

src\case_studies\control\pd.py

Generated: 2026-02-23 12:18:26

pd.py

src\case_studies\control\pd.py

TODO: should this inherit from ControllerBase and implement ff term + saturation?

class PD:

"""

PD (Proportional-Derivative) controller helper class.

NOTE: This is a HELPER class used inside full controller implementations (like ArmControllerPD). You can either write this once, and then use it in each controller (e.g. D_mass/pd_controller.py), or you can implement PD control directly inside each controller class instead. You don't need both.

This class implements the core PD control law:

- Regular PD: $u = kp \cdot error + kd \cdot error_dot$*
- Modified PD: $u = kp \cdot error - kd \cdot ydot$ (more common, avoids derivative kick)*

The derivative of y (or e) must be calculated outside of this class and passed in to the update methods. This is typically done using the DirtyDerivativeFilter or from direct state access.

"""

def __init__(self, kp: float, kd: float):

self.kp = kp

self.kd = kd

self.ydot = 0.0 *# Store last ydot for plotting*

def update(self, y_ref: float, y: float, error_dot: float):

"""

Regular PD control: Returns $u = kp \cdot error + kd \cdot error_dot$

Less common than modified PD (see update_modified).

Args:

y_ref (float): reference value for signal y.

y (float): current value of signal y.

error_dot (float): derivative of error.

Returns:

u (float): control output

"""

error = y_ref - y

u = self.kp * error + self.kd * error_dot

return u

```

def update_modified(self, y_ref: float, y: float, ydot: float):
    """
    Modified PD control: Returns  $u = k_p \text{error} - k_d \text{ydot}$ 

    This is the PREFERRED form of PD control because it does not add an extra zero
    the numerator of the closed-loop transfer function.

    Args:
        y_ref (float): reference value for signal y.
        y (float): current value of signal y.
        ydot (float): current derivative of signal y.

    Returns:
        u (float): control output
    """
    error = y_ref - y
    self.ydot = ydot # Store for plotting
    u = self.kp * error - self.kd * ydot
    return u

```